

NSI

Terminale

Édition de travail 2026-2027

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

Sommaire

1	Algorithmes sur les arbres et les graphes	1
1.1	Taille, hauteur et parcours d'un arbre binaire	1
1.2	Arbre binaire de recherche	2
1.3	Parcours d'un graphe	3
1.4	Cycle et chemin dans un graphe	4
1.5	Diviser pour régner	5
1.6	Programmation dynamique	6
1.7	Recherche d'un motif : l'algorithme de Boyer-Moore	7
1.8	Taille, hauteur et parcours d'un arbre binaire	8
1.9	Arbre binaire de recherche	10
1.10	Parcours d'un graphe	11
1.11	Cycle et chemin dans un graphe	12
1.12	Diviser pour régner	13
1.13	Programmation dynamique	14
1.14	Recherche d'un motif : l'algorithme de Boyer-Moore	15
1.15	Taille, hauteur et parcours d'un arbre binaire	16
1.16	Arbre binaire de recherche	18
1.17	Parcours d'un graphe	19
1.18	Cycle et chemin dans un graphe	20
1.19	Diviser pour régner	21
1.20	Programmation dynamique	22
2	Architectures matérielles, systèmes d'exploitation et réseaux	51
2.1	Le système sur puce	51
2.2	Processus : création, ordonnancement, interblocage	52
2.3	Protocoles de routage	53
2.4	Chiffrement symétrique et asymétrique	54
2.5	Le système sur puce	55
2.6	Processus : création, ordonnancement, interblocage	55
2.7	Protocoles de routage	57
2.8	Chiffrement symétrique et asymétrique	57
2.9	Le système sur puce	58
2.10	Processus : création, ordonnancement, interblocage	59
2.11	Protocoles de routage	60

3.1	Le modèle relationnel	81
3.2	Le système de gestion de bases de données	82
3.3	Interroger une base : SELECT, FROM, WHERE	83
3.4	Croiser les données : JOIN et ORDER BY	83
3.5	Modifier une base : INSERT, UPDATE, DELETE	84
3.6	Le modèle relationnel	85
3.7	Le système de gestion de bases de données	86
3.8	Interroger une base : SELECT, FROM, WHERE	87
3.9	Croiser les données : JOIN et ORDER BY	88
3.10	Modifier une base : INSERT, UPDATE, DELETE	88
3.11	Le modèle relationnel	89
3.12	Le système de gestion de bases de données	90
3.13	Interroger une base : SELECT, FROM, WHERE	91
3.14	Croiser les données : JOIN et ORDER BY	92
3.15	Modifier une base : INSERT, UPDATE, DELETE	92
3.16	Le modèle relationnel	93
3.17	Le système de gestion de bases de données	94

4.1	Événements clés de l'histoire de l'informatique	123
4.2	Évolution des rôles du logiciel et du matériel	123
4.3	Événements clés de l'histoire de l'informatique	124
4.4	Évolution des rôles du logiciel et du matériel	125
4.5	Événements clés de l'histoire de l'informatique	131
4.6	Évolution des rôles du logiciel et du matériel	132

5.1	Programme, donnée et calculabilité	137
5.2	Écrire et analyser un programme récursif	138
5.3	API, bibliothèques et modules	139
5.4	Paradigmes de programmation	140
5.5	Mise au point : causes typiques de bugs	141
5.6	Programme, donnée et calculabilité	142
5.7	Écrire et analyser un programme récursif	143
5.8	API, bibliothèques et modules	144
5.9	Paradigmes de programmation	145
5.10	Mise au point : causes typiques de bugs	146
5.11	Programme, donnée et calculabilité	147

5.12	Écrire et analyser un programme récursif	148
5.13	API, bibliothèques et modules	149
5.14	Paradigmes de programmation	150
5.15	Mise au point : causes typiques de bugs	151
5.16	Programme, donnée et calculabilité	152

6**Structures de données****185**

6.1	Interface et implémentation d'une structure de données	185
6.2	Définir et utiliser une classe en Python	186
6.3	Piles, files, et choix de structure adaptée	187
6.4	Arbres binaires	188
6.5	Graphes : modélisation et représentations	189
6.6	Interface et implémentation d'une structure de données	190
6.7	Définir et utiliser une classe en Python	191
6.8	Piles, files, et choix de structure adaptée	192
6.9	Arbres binaires	192
6.10	Graphes : modélisation et représentations	193
6.11	Interface et implémentation d'une structure de données	194
6.12	Définir et utiliser une classe en Python	196
6.13	Piles, files, et choix de structure adaptée	196
6.14	Arbres binaires	197
6.15	Graphes : modélisation et représentations	198
6.16	Interface et implémentation d'une structure de données	199
6.17	Définir et utiliser une classe en Python	200
6.18	Piles, files, et choix de structure adaptée	201
6.19	Arbres binaires	202

1 Algorithmes sur les arbres et les graphes

1.1 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0.

Exemple.

```
1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))
```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```
1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)
```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10            file.append(noeud.gauche)
11            if noeud.droite is not None:
12                file.append(noeud.droite)
13    return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.2 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7 def rechercher(arbre, cle):
8     if arbre is None:
9         return False
10    if cle == arbre.valeur:
11        return True
12    if cle < arbre.valeur:
13        return rechercher(arbre.gauche, cle)
14    return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```
1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre
```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.3 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
```

```
10 return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4     visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])`: au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.4 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7     elif voisin != parent:
```



```

8         return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets A et B revient à effectuer un parcours (BFS ou DFS) depuis A en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis B pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si B n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                predecesseur[voisin] = sommet
11                file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
14     chemin_trouve = []
15     sommet = arrivee
16     while sommet is not None:
17         chemin_trouve.append(sommet)
18         sommet = predecesseur[sommet]
19     chemin_trouve.reverse()
20     return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.5 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):

```

```

5         if gauche[i] ≤ droite[j]:
6             resultat.append(gauche[i])
7             i += 1
8         else:
9             resultat.append(droite[j])
10            j += 1
11    return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) ≤ 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)

```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

✦ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.6 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémorise** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```

1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p ≤ somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)

```

```

8         if meilleur is None or candidat < meilleur:
9             meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p ≤ somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémoïsation** Sans mémoïsation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémoïsation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.7 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1 def recherche_naive(texte, motif):
2     positions = []
3     for i in range(len(texte) - len(motif) + 1):
4         if texte[i:i + len(motif)] == motif:
5             positions.append(i)
6     return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1 def table_decalage(motif):
2     table = {}
3     for i, c in enumerate(motif):
4         table[c] = i
5     return table
6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i ≤ n - m:
13        j = m - 1
14        while j ≥ 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions

```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

1.8 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):

```

```

3     self.valeur = valeur
4     self.gauche = gauche
5     self.droite = droite

```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0.

Exemple.

```

1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))

```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```

1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)

```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:

```

```

10     file.append(noeud.gauche)
11     if noeud.droite is not None:
12         file.append(noeud.droite)
13     return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/-suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.9 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7     def rechercher(arbre, cle):
8         if arbre is None:
9             return False
10        if cle == arbre.valeur:
11            return True
12        if cle < arbre.valeur:
13            return rechercher(arbre.gauche, cle)
14        return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```

1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre

```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de `inserer`. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.10 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```

1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
10    return visites

```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version réursive :

```

1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []

```

```

4     visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites

```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[]):` au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.11 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```

1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7         elif voisin != parent:
8             return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets *A* et *B* revient à effectuer un parcours (BFS ou DFS) depuis *A* en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis *B* pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si *B* n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}

```



```

3  file = [depart]
4  while file:
5      sommet = file.pop(0)
6      if sommet == arrivee:
7          break
8      for voisin in graphe[sommet]:
9          if voisin not in predecesseur:
10             predecesseur[voisin] = sommet
11             file.append(voisin)
12  if arrivee not in predecesseur:
13      return None
14  chemin_trouve = []
15  sommet = arrivee
16  while sommet is not None:
17      chemin_trouve.append(sommet)
18      sommet = predecesseur[sommet]
19  chemin_trouve.reverse()
20  return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.12 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1  def fusionner(gauche, droite):
2      resultat = []
3      i, j = 0, 0
4      while i < len(gauche) and j < len(droite):
5          if gauche[i] <= droite[j]:
6              resultat.append(gauche[i])
7              i += 1
8          else:
9              resultat.append(droite[j])
10             j += 1
11  return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)

```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.13 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recourent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémoise** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```
1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
8             if meilleur is None or candidat < meilleur:
9                 meilleur = candidat
10    return meilleur
```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```
1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
```

```

7   for p in pieces:
8       if p ≤ somme:
9           candidat = 1 + rendu_memo(pieces, somme - p, memo)
10          if meilleur is None or candidat < meilleur:
11              meilleur = candidat
12      memo[somme] = meilleur
13      return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémorisation** Sans mémorisation, le nombre d'appels récur­sifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémorisation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.14 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1 def recherche_naive(texte, motif):
2     positions = []
3     for i in range(len(texte) - len(motif) + 1):
4         if texte[i:i + len(motif)] == motif:
5             positions.append(i)
6     return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1 def table_decalage(motif):
2     table = {}
3     for i, c in enumerate(motif):
4         table[c] = i
5     return table

```

```

6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i ≤ n - m:
13        j = m - 1
14        while j ≥ 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions

```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

1.15 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite

```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0.

Exemple.

```

1 def taille(arbre):

```

```

2   if arbre is None:
3       return 0
4   return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))

```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```

1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)

```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10             file.append(noeud.gauche)
11         if noeud.droite is not None:
12             file.append(noeud.droite)
13     return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.16 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7     def rechercher(arbre, cle):
8         if arbre is None:
9             return False
10        if cle == arbre.valeur:
11            return True
12        if cle < arbre.valeur:
13            return rechercher(arbre.gauche, cle)
14        return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```

1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre

```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche`

vaut None : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.17 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
10    return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4     visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])` : au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la

définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement visites. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.18 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7         elif voisin != parent:
8             return True
9     return False
```

DÉFINITION 2

Chercher un chemin entre deux sommets *A* et *B* revient à effectuer un parcours (BFS ou DFS) depuis *A* en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis *B* pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si *B* n'est jamais atteint, aucun chemin n'existe.

```
1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                 predecesseur[voisin] = sommet
11                 file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
```



```

14     chemin_trouve = []
15     sommet = arrivee
16     while sommet is not None:
17         chemin_trouve.append(sommet)
18         sommet = predecesseur[sommet]
19     chemin_trouve.reverse()
20     return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester if arrivee not in predecesseur, la reconstruction du chemin lève une KeyError lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici None) plutôt que de planter.

1.19 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** les solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1  def fusionner(gauche, droite):
2      resultat = []
3      i, j = 0, 0
4      while i < len(gauche) and j < len(droite):
5          if gauche[i] <= droite[j]:
6              resultat.append(gauche[i])
7              i += 1
8          else:
9              resultat.append(droite[j])
10             j += 1
11     return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)

```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.20 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recourent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémoise** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```

1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p ≤ somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
8             if meilleur is None or candidat < meilleur:
9                 meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p ≤ somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémorisation** Sans mémorisation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémorisation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 2 ◆

En utilisant les fonctions `inserer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 3 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 4 ◆◆

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémorisation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 5 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 6 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 7 ♦

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 8 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 9 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 10 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 11 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 12 ◆

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 13 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 14 ◆◆

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 15 ◆◆

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 16 ◆

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 17 ◆

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 18 ◆◆

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
```

```

5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 19

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 20

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 21

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 22

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 23

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 24

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 25 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 26 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 27 ♦

En utilisant les fonctions `inserer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 28 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 29 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 30 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 31 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 32

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 33

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 34

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 35

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 36

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 37

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 38

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
```



```

5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 39 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 40 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 41 ♦

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 42 ♦

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 43 ♦♦

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```

1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }

```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 44 ♦♦

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 45

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

Exercice 46

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 47

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 48

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 49

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 50

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

QCM d'auto-évaluation — Algorithmique

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C01B/C01D — Arbres

1. [Q1] La hauteur d'un arbre réduit à un seul nœud (une feuille) vaut :
 - A. -1.
 - B. 0.
 - C. 1.
 - D. indéfinie.
2. [Q2] Un parcours en largeur d'un arbre utilise :
 - A. une pile.
 - B. une file.
 - C. un dictionnaire uniquement.
 - D. aucune structure auxiliaire.

C02B/C02C — Graphes

3. [Q3] Le parcours en profondeur d'un graphe (DFS) s'implémente naturellement par :
 - A. récursion (ou une pile explicite).
 - B. une file uniquement.
 - C. un tri préalable des sommets.
 - D. une recherche dichotomique.
4. [Q4] Lors d'un parcours d'un graphe non orienté, on détecte un cycle lorsque l'on rencontre un sommet déjà visité qui :
 - A. est le sommet de départ.
 - B. n'est pas le parent direct dans le parcours.
 - C. a le plus grand nombre de voisins.
 - D. n'a aucun voisin.

C04/C05 — Programmation dynamique, recherche de motif

5. [Q5] La programmation dynamique est particulièrement utile lorsque :
 - A. les sous-problèmes ne se recoupent jamais.
 - B. les mêmes sous-problèmes réapparaissent plusieurs fois.
 - C. le problème n'a pas de solution récursive.
 - D. il n'y a qu'un seul sous-problème.
6. [Q6] Contrairement à la recherche naïve, l'algorithme de Boyer-Moore compare le motif au texte :
 - A. de droite à gauche, en s'appuyant sur un prétraitement du motif.
 - B. en ignorant totalement le texte.
 - C. uniquement sur le premier caractère.
 - D. dans un ordre aléatoire.

Corrigé

Q1. L'arbre a 5 nœuds (10, 5, 15, 2, 7) : taille = 5. Le plus long chemin racine-feuille est $10 \rightarrow 5 \rightarrow 2$ (ou $10 \rightarrow 5 \rightarrow 7$), soit 2 arêtes : hauteur = 2.

Q2.

```
1 def est_feuille(arbre):
2     return arbre is not None and arbre.gauche is None and arbre.droite is None
```

Corrigé

Q1. L'ABR obtenu a pour racine 10, sous-arbre gauche enraciné en 5 (avec 3 à gauche et 7 à droite), et sous-arbre droit enraciné en 15 (avec 12 à gauche).

Q2.

```
1 def minimum(arbre):
2     while arbre.gauche is not None:
3         arbre = arbre.gauche
4     return arbre.valeur
```

Dans un ABR, chaque descente à gauche mène vers des valeurs plus petites ; la plus petite valeur est donc atteinte lorsqu'il n'y a plus de sous-arbre gauche.

Évaluation A — Arbres binaires

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (taille, hauteur) ; Ex. 2 : 12 pts (arbre de recherche)

Exercice 1 — Taille et hauteur

(8 points)

On donne la classe Noeud du cours (attributs valeur, gauche, droite) et l'arbre :

```
1 arbre = Noeud(10, Noeud(5, Noeud(2), Noeud(7)), Noeud(15))
```

1. (4 pts) Sans exécuter de code, donner la taille et la hauteur de cet arbre en justifiant (convention : hauteur d'une feuille = 0).
2. (4 pts) Écrire une fonction `est_feuille(arbre)` qui renvoie True si et seulement si arbre est un nœud sans aucun enfant.

Exercice 2 — Arbre binaire de recherche

(12 points)

1. (6 pts) En utilisant `insérer` du cours, construire un ABR à partir des valeurs 10, 5, 15, 3, 7, 12 insérées dans cet ordre.
2. (6 pts) Écrire une fonction `minimum(arbre)` qui renvoie la plus petite valeur d'un ABR non vide (indication : la plus petite valeur d'un ABR se trouve toujours tout en bas à gauche).

Corrigé

Q1. Depuis "P" : on enfile "P", on le défile et on enfile ses voisins "Q" puis "R" ; on défile "Q", son voisin "S" n'est pas encore visité, on l'enfile ; on défile "R", son voisin "S" est déjà visité. Ordre de visite : P, Q, R, S.

Q2. Oui : $P \rightarrow Q \rightarrow S \rightarrow R \rightarrow P$ est un cycle (chaque arête utilisée existe bien dans le graphe, et on revient au sommet de départ sans repasser par une même arête).

Corrigé

Q1. Diviser (découper le problème en sous-instances plus petites de même nature), régner (résoudre récursivement chaque sous-instance), combiner (assembler les solutions partielles en la solution globale).

Q2.

```
1 def somme_dpr(liste):
2     if len(liste) == 1:
```

```

3     return liste[0]
4     milieu = len(liste) // 2
5     return somme_dpr(liste[:milieu]) + somme_dpr(liste[milieu:])

```

Évaluation B — Graphes et diviser pour régner

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (parcours et cycle); Ex. 2 : 10 pts (diviser pour régner)

Exercice 1 — Parcours et cycle

(10 points)

On donne le graphe non orienté :

```

1 graphe = {
2     "P": ["Q", "R"],
3     "Q": ["P", "S"],
4     "R": ["P", "S"],
5     "S": ["Q", "R"],
6 }

```

- (5 pts) Donner l'ordre de visite des sommets par un parcours en largeur (BFS) depuis "P", en supposant que les voisins sont explorés dans l'ordre où ils apparaissent dans la liste.
- (5 pts) Ce graphe contient-il un cycle ? Justifier en citant un cycle si oui.

Exercice 2 — Diviser pour régner

(10 points)

- (4 pts) Rappeler les trois étapes de la méthode diviser pour régner.
- (6 pts) Écrire une fonction récursive `somme_dpr(liste)` qui calcule la somme des éléments d'une liste non vide par diviser pour régner (diviser en deux moitiés, additionner récursivement, puis combiner par addition).

Exercice 51

Une fonction de mémorisation est écrite ainsi :

```

1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]

```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 52

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 53

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 54

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 55 ◆

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 56 ◆

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 57 ◆

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 58 ◆

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que $\text{memo}=\{\}$ recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement $\text{memo}=\text{None}$ suivi d'une initialisation à l'intérieur de la fonction.

Exercice 59 ◆

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut $\text{memo}=\{\}$ est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels

qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 60

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 61

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle `f(3)`, puis `f(2, {})` (avec un dictionnaire frais), puis `f(3)` à nouveau. Le deuxième appel à `f(3)` recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à `f(3)` retrouve directement 9 dans `memo`, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 62

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 63 ♦

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Exercice 64 ♦

Une fonction de mémorisation est écrite ainsi :

```
1 def f(n, memo={}):
2     if n in memo:
3         return memo[n]
4     memo[n] = n * n
5     return memo[n]
```

On appelle $f(3)$, puis $f(2, \{\})$ (avec un dictionnaire frais), puis $f(3)$ à nouveau. Le deuxième appel à $f(3)$ recalcule-t-il 3×3 , ou retrouve-t-il directement le résultat mémorisé du tout premier appel ? Pourquoi ce comportement peut-il surprendre ?

Corrigé

Le deuxième appel à $f(3)$ retrouve directement 9 dans memo, sans recalcul : le dictionnaire par défaut `memo={}` est créé **une seule fois**, à la définition de la fonction, et **partagé** par tous les appels qui ne fournissent pas explicitement leur propre dictionnaire. C'est surprenant car on pourrait croire, par analogie avec d'autres langages, que `memo={}` recrée un dictionnaire vide à **chaque** appel – ce n'est pas le cas en Python. C'est pourquoi le cours utilise systématiquement `memo=None` suivi d'une initialisation à l'intérieur de la fonction.

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])

```

```

6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récurifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
```

```

3     memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```


Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```
1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)
```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```

1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False

```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```

1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']

```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

```

1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]

```

Comme `rendu_memo`, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans `memo`. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```

1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)

```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).

Corrigé

```

1 def compter_feuilles(arbre):
2     if arbre is None:
3         return 0
4     if arbre.gauche is None and arbre.droite is None:
5         return 1
6     return compter_feuilles(arbre.gauche) + compter_feuilles(arbre.droite)

```

Cas de base : l'arbre vide n'a aucune feuille. Un nœud sans aucun enfant est lui-même une feuille (on renvoie 1, sans appel récursif supplémentaire). Sinon, on additionne le nombre de feuilles des deux sous-arbres.

Corrigé

```
1 abr = None
2 for v in (7, 3, 9, 1, 5):
3     abr = inserer(abr, v)
4
5 print(rechercher(abr, 5)) # True
6 print(rechercher(abr, 4)) # False
```

L'arbre construit a pour racine 7, sous-arbre gauche enraciné en 3 (lui-même avec 1 à gauche et 5 à droite), et sous-arbre droit réduit à 9. La recherche de 5 descend $7 \rightarrow 3 \rightarrow 5$ (trouvé) ; celle de 4 descend $7 \rightarrow 3 \rightarrow 5 \rightarrow$ sous-arbre gauche vide (non trouvé).

Corrigé

```
1 trajet = chemin(metro, "A", "E")
2 print(trajet) # ['A', 'B', 'C', 'E']
```

Le BFS depuis "A" atteint "B" en premier (distance 1), puis "C" et "D" (distance 2, "C" découvert avant "D" car il apparaît en premier dans la liste des voisins de "B"), puis "E" (distance 3, découvert depuis "C"). Le trajet reconstruit est $A \rightarrow B \rightarrow C \rightarrow E$: il compte 4 stations (départ et arrivée compris).

Corrigé

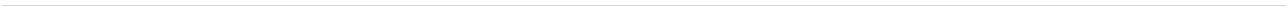
```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {0: 0, 1: 1}
4     if n in memo:
5         return memo[n]
6     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
7     return memo[n]
```

Comme rendu_memo, les cas de base ($n = 0$ et $n = 1$) sont préchargés dans memo. Sans mémorisation, le nombre d'appels récursifs pour calculer F_{30} dépasse le million ; avec mémorisation, chaque valeur $F_0, \dots, F_{30}$ n'est calculée qu'une seule fois.

Corrigé

```
1 def maximum_dpr(liste):
2     if len(liste) == 1:
3         return liste[0]
4     milieu = len(liste) // 2
5     max_gauche = maximum_dpr(liste[:milieu])
6     max_droite = maximum_dpr(liste[milieu:])
7     return max(max_gauche, max_droite)
```

Le cas de base est une liste réduite à un seul élément : son maximum est cet élément lui-même, sans appel récursif. On suppose la liste non vide en entrée (une liste vide n'a pas de maximum).



2 Architectures matérielles, systèmes d'exploitation et réseaux

2.1 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

+ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.2 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution
```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```
1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}
```


◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.3 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2     "A": {"B": 1, "C": 5},
3     "B": {"A": 1, "D": 1},
4     "C": {"A": 5, "D": 1},
5     "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.4 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer **et** déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def déchiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

◆ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

✚ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

2.5 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

+ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.6 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue

le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```

1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution

```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```

1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}

```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un cycle dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.7 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2   "A": {"B": 1, "C": 5},
3   "B": {"A": 1, "D": 1},
4   "C": {"A": 5, "D": 1},
5   "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.8 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer et déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def dechiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

◆ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

+ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

2.9 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

✦ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.10 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
```

```

9     temps_restant[p] -= execute
10    if temps_restant[p] > 0:
11        file.append(p)
12    return ordre_execution

```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```

1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}

```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.11 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2     "A": {"B": 1, "C": 5},
3     "B": {"A": 1, "D": 1},
4     "C": {"A": 5, "D": 1},
5     "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 2 ◆◆

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 3 ◆◆

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 4

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 5

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 6

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 7

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 8

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 9

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 10

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 11 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 12 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 13 ♦

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 14 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 15 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 16 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 17

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 18

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 19

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 20

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 21

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 22

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 23

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 24 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 25 ♦

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 26 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 27 ♦♦

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 28 ♦

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 29 ♦

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 30 ♦♦

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 31

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 32

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 33

En utilisant la fonction `tourniquet` du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 34

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 35

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 36

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 37

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 38

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 39

On donne le réseau de coûts suivant :

```
1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }
```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant `bfs_sauts` du cours, combien de sauts compte-t-elle ?
2. En utilisant `plus_court_cout` du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 40

En utilisant `chiffrer_xor` du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

Exercice 41

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 42

Trois processus se partagent trois ressources : P1 retient R1 et attend R2 ; P2 retient R2 et attend R3 ; P3 retient R3 et attend R1.

1. Représenter cette situation par les deux dictionnaires `attente` et `retenue_par` du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

QCM d'auto-évaluation — Architectures, systèmes, réseaux

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C01/C02A — Système sur puce, processus

1. [Q1] L'inconvénient principal d'un système sur puce (SoC) par rapport à des composants séparés est :
 - A. une consommation énergétique plus élevée.
 - B. la perte de modularité (impossible de remplacer un seul composant défaillant).
 - C. une taille plus importante.
 - D. un coût de fabrication toujours plus élevé.
2. [Q2] Créer un processus consiste, pour le système d'exploitation, à :
 - A. lui allouer de la mémoire et l'inscrire dans la liste des processus à exécuter.
 - B. éteindre tous les autres processus.
 - C. compiler son code source.
 - D. lui attribuer une adresse IP.

C02C — Interblocage

3. [Q3] Un interblocage se traduit, dans le graphe d'attente, par :
 - A. un sommet isolé.
 - B. un cycle.
 - C. un arbre.
 - D. un graphe vide.

C03 — Routage

4. [Q4] Le protocole RIP choisit la route qui minimise :
 - A. le coût cumulé des liaisons.
 - B. le nombre de sauts.
 - C. la latence mesurée en temps réel.
 - D. le nombre d'utilisateurs connectés.

C04A/C04B — Chiffrement, HTTPS

5. [Q5] Le chiffrement asymétrique utilise :
 - A. une seule clé secrète partagée.
 - B. une paire de clés, l'une publique et l'une privée.
 - C. aucune clé.
 - D. une clé différente à chaque caractère.
6. [Q6] Lors d'une connexion HTTPS, le chiffrement asymétrique sert principalement à :
 - A. chiffrer l'intégralité des données échangées de bout en bout.
 - B. échanger en toute sécurité une clé symétrique de session.
 - C. afficher le cadenas dans le navigateur.
 - D. compresser les données.

Corrigé

Q1. Avantages : compacité, consommation énergétique réduite, coût de fabrication réduit en grande série. Inconvénient : perte de modularité (un composant défaillant oblige à remplacer toute la puce).

Q2. Le système d'exploitation alloue de la mémoire au nouveau processus, y charge le code du programme, et l'inscrit dans la liste des processus à exécuter.

Corrigé

Q1.

```
1 resultat = tourniquet(["P1", "P2", "P3"], 2, {"P1": 2, "P2": 4, "P3": 3})
2 print(resultat) # [('P1', 2), ('P2', 2), ('P3', 2), ('P2', 2), ('P3', 1)]
```

P1 termine en une seule tranche (durée 2), P2 et P3 nécessitent chacun deux passages.

Q2. Non, il n'y a **pas** d'interblocage : P3 retient R3 sans rien attendre, donc la chaîne d'attente P1 → P2 (via R2 puis R1) ne boucle pas – P2 finira par obtenir R1 dès que P1 le libère, sans dépendre de personne d'autre.

Évaluation A — Système sur puce et processus

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (SoC, création de processus); Ex. 2 : 12 pts (ordonnancement, interblocage)

Exercice 1 — Système sur puce et processus

(8 points)

1. (4 pts) Citer trois avantages de l'intégration de plusieurs composants sur un système sur puce, et son principal inconvénient.
2. (4 pts) Décrire, en une phrase, ce que fait le système d'exploitation lors de la création d'un processus.

Exercice 2 — Ordonnancement et interblocage

(12 points)

1. (6 pts) En utilisant tourniquet du cours, déterminer l'ordre d'exécution pour trois processus P1, P2, P3 de durées respectives 2, 4, 3, avec une tranche de temps de 2.
2. (6 pts) Trois processus P1, P2, P3 retiennent chacun une ressource R1, R2, R3, et P1 attend R2, P2 attend R1 (pas de troisième attente). Y a-t-il un interblocage ? Justifier avec a_un_interblocage du cours.

Corrigé

Q1. RIP choisirait la route directe A-D (un seul saut), quel que soit son coût élevé. bfs_sauts(reseau, "A", "D") renvoie 1.

Q2. plus_court_cout(reseau, "A", "D") renvoie 3 : la route la moins coûteuse est A-B-C-D (coût $1 + 1 + 1 = 3$), bien moins chère que la route directe (coût 8). Elle **ne coïncide pas** avec la route RIP : RIP privilégie le nombre de sauts, OSPF le coût cumulé.

Corrigé

Q1. Le chiffrement symétrique utilise une **unique clé secrète** partagée pour chiffrer et déchiffrer ; le chiffrement asymétrique utilise une **paire de clés** (publique/privée) : ce qui est chiffré avec la clé publique ne se déchiffre qu'avec la clé privée correspondante.

Q2. Le chiffrement asymétrique est beaucoup plus lent à calculer que le chiffrement symétrique. L'utiliser pour **tout** l'échange (pages web, formulaires, fichiers) ralentirait considérablement la connexion. HTTPS l'utilise donc seulement pour échanger, en toute sécurité, une clé symétrique de session au début de la connexion ; le reste des données est ensuite chiffré avec cette clé symétrique, bien plus rapide.

Évaluation B — Routage et cryptographie

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (routage) ; Ex. 2 : 10 pts (chiffrement, HTTPS)

Exercice 1 — Routage

(10 points)

```
1 reseau = {
2     "A": {"B": 1, "D": 8},
3     "B": {"A": 1, "C": 1},
4     "C": {"B": 1, "D": 1},
5     "D": {"A": 8, "C": 1},
6 }
```

- (5 pts) En utilisant `bfs_sauts` du cours, quelle route RIP choisirait-il de A vers D, et en combien de sauts ?
- (5 pts) En utilisant `plus_court_cout` du cours, quel est le coût de la route la moins coûteuse de A vers D ? Coïncide-t-elle avec la route RIP ?

Exercice 2 — Chiffrement et HTTPS

(10 points)

- (4 pts) Quelle est la différence essentielle entre chiffrement symétrique et asymétrique ?
- (6 pts) Expliquer pourquoi une connexion HTTPS combine les deux, plutôt que d'utiliser uniquement le chiffrement asymétrique pour tout l'échange.

Exercice 43

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 44

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 45

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 46

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 47

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 48

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 49

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Exercice 50 ◆

Un ordinateur possède un processeur à un seul cœur, et pourtant l'utilisateur peut écouter de la musique tout en tapant du texte dans un traitement de texte, sans interruption perceptible. Ces deux tâches s'exécutent-elles en parallélisme réel ou en concurrence ? Expliquer.

Corrigé

Ces deux tâches s'exécutent en **concurrence**, pas en parallélisme réel : un seul cœur ne peut exécuter qu'une instruction à la fois. L'ordonnanceur alterne très rapidement entre les deux processus (tranches de temps de quelques millisecondes), ce qui donne à l'utilisateur l'**illusion** d'une exécution simultanée, alors qu'en réalité chaque processus n'avance que pendant sa tranche de temps.

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente P1 → P2 → P3 → P1 signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. bfs_sauts(reseau, "A", "C") renvoie 1.

Q2. plus_court_cout(reseau, "A", "C") renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** cle sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```


Ce chiffrement est dit **symétrique** car la **même clé** `c` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente P1 → P2 → P3 → P1 signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût 2+2 = 4), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `c` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```
1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True
```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```
1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]
```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

Q1.

```
1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}
```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.

Corrigé

Q1. RIP choisirait la route directe A-C : elle ne compte qu'un seul saut, quel que soit son coût. `bfs_sauts(reseau, "A", "C")` renvoie 1.

Q2. `plus_court_cout(reseau, "A", "C")` renvoie 4 : la route la moins coûteuse est A-B-C (coût $2+2=4$), moins chère que la route directe A-C (coût 10), bien qu'elle traverse un routeur de plus. OSPF choisirait donc A-B-C, RIP choisirait A-C : les deux protocoles divergent car ils n'optimisent pas le même critère.

Corrigé

```

1 message = b"BAC2027"
2 cle = b"NSI"
3 chiffre = chiffrer_xor(message, cle)
4 dechiffre = chiffrer_xor(chiffre, cle)
5 print(dechiffre == message) # True

```

Ce chiffrement est dit **symétrique** car la **même clé** `cle` sert à la fois à chiffrer et à déchiffrer : il n'y a qu'un seul secret partagé, contrairement au chiffrement asymétrique qui utilise deux clés différentes (publique et privée).

Corrigé

```

1 resultat = tourniquet(["X", "Y"], 3, {"X": 4, "Y": 6})
2 print(resultat) # [('X', 3), ('Y', 3), ('X', 1), ('Y', 3)]

```

X s'exécute 3 puis 1 unité de temps (total 4), Y s'exécute 3 puis 3 unités de temps (total 6). Il faut 4 tranches de temps au total pour que les deux processus se terminent.

Corrigé

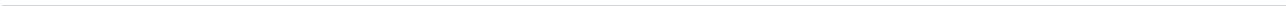
Q1.

```

1 attente = {"P1": "R2", "P2": "R3", "P3": "R1"}
2 retenue_par = {"R1": "P1", "R2": "P2", "R3": "P3"}

```

Q2. Oui, il y a interblocage : P1 attend R2 retenue par P2, qui attend R3 retenue par P3, qui attend R1 retenue par P1 – on revient au point de départ. Le cycle d'attente $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$ signifie qu'aucun des trois processus ne peut jamais obtenir la ressource qu'il attend.



3 Bases de données relationnelles

3.1 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, annee)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

✚ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.2 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.3 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, annee INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels ($=$, $<$, $>$, $\leq$, $\geq$, $\neq$ pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE annee > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.4 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe **JOIN ... ON ...** précise la condition de rapprochement.

Exemple. Reprenons Livre et une table Emprunt qui référence Livre.id :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
```

```

2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;

```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table Emprunt.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause ORDER BY trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (ASC, par défaut) ou décroissant (DESC).

Exemple. Livres triés du plus récent au plus ancien :

```

1 SELECT titre, annee FROM Livre ORDER BY annee DESC;

```

⚠ ERREUR FRÉQUENTE

Oublier que **ORDER BY** se place après **WHERE**. L'ordre des clauses est fixe : **SELECT ... FROM ... WHERE ... ORDER BY** Écrire **ORDER BY** avant **WHERE** est une erreur de syntaxe.

3.5 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec **VALUES**.

Exemple.

```

1 INSERT INTO Livre(titre, auteur, annee)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);

```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause **UPDATE ... SET ... WHERE ...** modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```

1 UPDATE Livre SET annee = 1954 WHERE titre = 'Fahrenheit 451';

```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```

1 DELETE FROM Livre WHERE annee < 1950;

```


⚠ ERREUR FRÉQUENTE

Exécuter un **UPDATE** ou un **DELETE** sans **WHERE**. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un **SELECT**.

» **Python À la machine.** Avant tout **UPDATE** ou **DELETE**, exécute d'abord le **SELECT** correspondant avec la même clause **WHERE** pour vérifier quels tuples seront affectés.

3.6 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, annee)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans `Emprunt`, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

♦ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes

les lignes concernées, au risque d'incohérences. Séparer Livre, Usager et Emprunt en trois relations liées par des clés étrangères élimine cette duplication.

✚ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.7 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément

pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.8 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, annee INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels ($=$, $<$, $>$, $\leq$, $\geq$, $\neq$ pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE annee > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.9 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe `JOIN ... ON ...` précise la condition de rapprochement.

Exemple. Reprenons `Livre` et une table `Emprunt` qui référence `Livre.id` :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table `Emprunt`.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause `ORDER BY` trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (`ASC`, par défaut) ou décroissant (`DESC`).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, annee FROM Livre ORDER BY annee DESC;
```

⚠ ERREUR FRÉQUENTE

Oublier que `ORDER BY` se place après `WHERE`. L'ordre des clauses est fixe : `SELECT ... FROM ... WHERE ... ORDER BY ...`. Écrire `ORDER BY` avant `WHERE` est une erreur de syntaxe.

3.10 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec `VALUES`.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, annee)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause `UPDATE ... SET ... WHERE ...` modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause `WHERE`, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET annee = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE annee < 1950;
```

⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

3.11 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, annee)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans `Emprunt`, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.12 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.13 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : SELECT (quels attributs afficher), FROM (dans quelle relation), et optionnellement WHERE (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, annee INTEGER);
2 INSERT INTO Livre VALUES
3   (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4   (2, '1984', 'Orwell', 1949),
5   (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause WHERE ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, ≤, ≥, ≠ pour différent) se combinent avec AND, OR, NOT pour des conditions composées ; LIKE permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE annee > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. `WHERE auteur = Orwell` est une erreur : SQL interprète `Orwell` sans guillemets comme un nom de colonne inexistant. Il faut écrire `WHERE auteur = 'Orwell'`, avec des apostrophes simples autour du texte.

3.14 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe `JOIN ... ON ...` précise la condition de rapprochement.

Exemple. Reprenons `Livre` et une table `Emprunt` qui référence `Livre.id` :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table `Emprunt`.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause `ORDER BY` trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (`ASC`, par défaut) ou décroissant (`DESC`).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, annee FROM Livre ORDER BY annee DESC;
```

⚠ ERREUR FRÉQUENTE

Oublier que **ORDER BY** se place après **WHERE**. L'ordre des clauses est fixe : `SELECT ... FROM ... WHERE ... ORDER BY ....` Écrire `ORDER BY` avant `WHERE` est une erreur de syntaxe.

3.15 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec `VALUES`.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, annee)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```


◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause **UPDATE ... SET ... WHERE ...** modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET annee = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE annee < 1950;
```

⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

3.16 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, annee)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.17 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
```

```

8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.

```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

Méthodes

► Méthode directe de travail sur le sujet.

Exercice 1 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```

1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)

```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 2 ◆

On dispose de la table suivante :

```

1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 3 ◆◆

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 4 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 5 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 6 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 7 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 8 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 9 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 10 ♦

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 11 ♦♦

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 12 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 13 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 14

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 15

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 16

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 17

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 18 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 19 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 20 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 21 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 22 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 23

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 24

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 25

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 26

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 27

On dispose de deux tables :


```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 28 ♦♦

On dispose de la table :

```

1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);

```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 29 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```

1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)

```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 30 ♦

On dispose de la table suivante :

```

1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 31 ♦♦

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 32 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 33

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 34

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 35

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 36

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 37 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 38 ◆

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 39 ◆◆

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 40 ◆◆

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 41 ◆

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?

- Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 42

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

- les titres des films réalisés par Miyazaki ;
- les titres des films dont la durée dépasse 100 minutes.

Exercice 43

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 44

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

- ajouter un produit 'Gomme' avec une quantité de 25 ;
- diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
- supprimer tous les produits dont la quantité en stock est nulle.

Exercice 45

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

- Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
- Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 46

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
```

```

2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 47 ♦♦

On dispose de deux tables :

```

1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)

```

où Note.id_eleve est une clé étrangère vers Eleve.id. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 48 ♦♦

On dispose de la table :

```

1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantite INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Regle', 12);

```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

Exercice 49 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```

1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)

```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 50 ♦

On dispose de la table suivante :

```

1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);

```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

QCM d'auto-évaluation — Bases de données

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C01A/C01C — Modèle relationnel

1. [Q1] Dans une relation, un attribut est défini sur :
 - A. un tuple.
 - B. un domaine (ensemble des valeurs possibles).
 - C. une clé étrangère uniquement.
 - D. une autre relation.
2. [Q2] Stocker le titre et l'auteur d'un livre à chaque ligne d'une table d'emprunts (au lieu d'une table Livre séparée) provoque :
 - A. un gain de temps de calcul systématique.
 - B. une duplication de données et un risque d'incohérence lors des mises à jour.
 - C. une amélioration de la sécurité.
 - D. aucun problème particulier.

C02 — Services d'un SGBD

3. [Q3] Le service de **persistance** d'un SGBD garantit que :
 - A. les données restent stockées durablement, même après la fin du programme.
 - B. les requêtes s'exécutent instantanément.
 - C. deux utilisateurs ne peuvent jamais accéder à la base en même temps.
 - D. les mots de passe sont chiffrés.

C03B/C03C — SELECT, WHERE

4. [Q4] La requête `SELECT nom FROM Client WHERE ville = 'Lyon'`; affiche :
 - A. toutes les colonnes des clients de Lyon.
 - B. uniquement le nom des clients habitant à Lyon.
 - C. le nombre de clients de Lyon.
 - D. toutes les villes des clients.
5. [Q5] Écrire `WHERE ville = Lyon` (sans apostrophes) au lieu de `WHERE ville = 'Lyon'` :
 - A. ne change rien au résultat.
 - B. est une erreur : Lyon est interprété comme un nom de colonne.
 - C. rend la requête plus rapide.
 - D. est obligatoire pour les textes courts.

C03D — JOIN

6. [Q6] La clause `JOIN ... ON A.x = B.y` permet de :
 - A. trier les résultats de la table A.
 - B. combiner les tuples de A et B dont les valeurs de x et y coïncident.
 - C. supprimer les doublons de A.
 - D. renommer la colonne x en y.

Corrigé

Q1. Une **relation** est une table regroupant des tuples décrits par les mêmes attributs. Un **attribut** est une colonne de la relation, définie sur un domaine. Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon unique chaque tuple.

Q2. `Commande.id_client` est une clé étrangère vers `Client.id` : elle permet de savoir quel client a passé chaque commande, sans dupliquer les informations du client dans la table `Commande`.

Corrigé

Q1.

```
1 SELECT nom FROM Produit WHERE categorie = 'Papeterie';
```

Q2.

```
1 SELECT nom FROM Produit WHERE prix < 2.00;
```

Q3.

```
1 INSERT INTO Produit(nom, categorie, prix) VALUES ('Gomme', 'Papeterie', 0.60);
```

Évaluation A — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 8 pts (modèle relationnel) ; Ex. 2 : 12 pts (SELECT, WHERE, INSERT)

Exercice 1 — Modélisation

(8 points)

- (4 pts) Définir, en une phrase chacun, les termes *relation*, *attribut* et *clé primaire*.
- (4 pts) Donner un exemple de clé étrangère dans le schéma `Commande(id, id_client, date) / Client(id, nom)`, et expliquer à quoi elle sert.

Exercice 2 — SELECT, WHERE, INSERT

(12 points)

On donne la table :

```
1 CREATE TABLE Produit(id INTEGER PRIMARY KEY, nom TEXT, prix REAL, categorie TEXT);
2 INSERT INTO Produit VALUES
3   (1, 'Cahier', 1.50, 'Papeterie'),
4   (2, 'Stylo', 0.80, 'Papeterie'),
5   (3, 'Casque audio', 25.00, 'Electronique');
```

- (4 pts) Écrire la requête affichant le nom des produits de catégorie 'Papeterie'.
- (4 pts) Écrire la requête affichant le nom des produits dont le prix est strictement inférieur à 2.00.
- (4 pts) Écrire la requête ajoutant un produit 'Gomme', catégorie 'Papeterie', prix 0.60.

Corrigé

Q1. Persistance, concurrence, efficacité, sécurisation.

Q2. Un fichier CSV n'offre aucune gestion de la **concurrence** : si deux guichets modifient le fichier en même temps, l'une des deux réservations peut être écrasée ou perdue. Un SGBD sérialise les accès concurrents pour garantir qu'aucune réservation n'est perdue.

Corrigé

Q1.

```
1 SELECT Employe.nom, Service.nom
2 FROM Employe
3 JOIN Service ON Employe.id_service = Service.id
4 ORDER BY Employe.nom ASC;
```

Q2.

```
1 UPDATE Employe SET id_service = 1 WHERE nom = 'Ben Ali';
```

Q3.

```
1 DELETE FROM Employe WHERE id_service = 1;
```

Après la question précédente, 'Ben Ali' et 'Chraibi' sont tous deux dans le service 1 : il ne reste que 'Haddad'. Sans clause WHERE, `DELETE FROM Employe;` supprimerait **tous** les employés, quel que soit leur service.

Évaluation B — Bases de données

Durée : 45 minutes. Ordinateur avec un SGBD (SQLite ou équivalent) autorisé.

Barème indicatif : Ex. 1 : 6 pts (SGBD); Ex. 2 : 14 pts (JOIN, ORDER BY, UPDATE, DELETE)

Exercice 1 — Services d'un SGBD

(6 points)

- (3 pts) Citer les quatre services rendus par un SGBD relationnel.
- (3 pts) Expliquer pourquoi un simple fichier CSV ne peut pas remplacer un SGBD pour gérer les réservations d'une salle de sport ouverte à plusieurs guichets simultanément.

Exercice 2 — JOIN, ORDER BY, UPDATE, DELETE

(14 points)

On donne :

```
1 CREATE TABLE Employe(id INTEGER PRIMARY KEY, nom TEXT, id_service INTEGER);
2 CREATE TABLE Service(id INTEGER PRIMARY KEY, nom TEXT);
3 INSERT INTO Service VALUES (1, 'Comptabilite'), (2, 'Informatique');
4 INSERT INTO Employe VALUES (1, 'Ben Ali', 2), (2, 'Chraibi', 1), (3, 'Haddad', 2);
```

- (5 pts) Écrire la requête affichant le nom de chaque employé avec le nom de son service, triée par nom d'employé croissant.
- (4 pts) Écrire la requête qui change le service de l'employé 'Ben Ali' vers le service 1.
- (5 pts) Écrire la requête qui supprime tous les employés du service 1 (après la modification de la question précédente). Que se passerait-il si l'on omettait la clause WHERE ?

Exercice 51 ◆

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 52 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 53 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 54 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 55 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 56 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 57

Une table Client contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 58

Une table Client contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 59

Une table Client contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 60

Une table Client contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause WHERE, DELETE supprime **tous** les tuples de la table. La table Client elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 61

Une table Client contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause WHERE). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 62 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Exercice 63 ♦

Une table `Client` contient 50 clients. On exécute la requête `DELETE FROM Client;` (sans clause `WHERE`). Combien de clients reste-t-il dans la table après cette requête ? La table elle-même existe-t-elle toujours ?

Corrigé

Il ne reste **aucun** client : sans clause `WHERE`, `DELETE` supprime **tous** les tuples de la table. La table `Client` elle-même continue d'exister (son schéma n'est pas touché), mais elle est vide : `SELECT COUNT(*) FROM Client;` renverrait 0.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;

```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;

```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise quantite - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantite = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 | SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 | SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 | SELECT Eleve.nom, Note.matiere, Note.valeur
2 | FROM Note
3 | JOIN Eleve ON Note.id_eleve = Eleve.id
4 | ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 | INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2 |
3 | UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4 |
5 | DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise quantite - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantite = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 | Adherent(id, nom, prenom)
2 | Sport(id, nom, coach)
3 | Seance(id, id_adherent, id_sport, jour)
```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 | SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```


Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère id_eleve, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;

```

Le UPDATE utilise quantite - 5 pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément quantite = 0 : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

Seance.id_adherent est une clé étrangère vers Adherent.id, et Seance.id_sport une clé étrangère vers Sport.id. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```
1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;
```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis ORDER BY ... DESC trie le résultat de la note la plus haute à la plus basse.

Corrigé

```
1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;
```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```
1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)
```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```
1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';
```

Q2.

```
1 SELECT titre FROM Film WHERE duree > 100;
```

Corrigé

```

1 SELECT Eleve.nom, Note.matiere, Note.valeur
2 FROM Note
3 JOIN Eleve ON Note.id_eleve = Eleve.id
4 ORDER BY Note.valeur DESC;

```

Le JOIN associe chaque note à l'élève correspondant via la clé étrangère `id_eleve`, puis `ORDER BY ... DESC` trie le résultat de la note la plus haute à la plus basse.

Corrigé

```

1 INSERT INTO Stock(produit, quantite) VALUES ('Gomme', 25);
2
3 UPDATE Stock SET quantite = quantite - 5 WHERE produit = 'Cahier';
4
5 DELETE FROM Stock WHERE quantite = 0;

```

Le UPDATE utilise `quantite - 5` pour exprimer la nouvelle valeur **relativement** à l'ancienne, sans avoir besoin de connaître ni réécrire la valeur numérique finale (35). Le DELETE cible précisément `quantite = 0` : seul 'Stylo' est supprimé.

Corrigé

Q1. Un adhérent pratiquant deux sports apparaît **deux fois** dans la table (une ligne par couple adhérent/sport), avec son nom et son prénom **dupliqués**. Si l'on corrige une faute dans son nom sur une seule ligne, les deux lignes deviennent incohérentes entre elles : c'est une anomalie de mise à jour.

Q2. On sépare en trois relations :

```

1 Adherent(id, nom, prenom)
2 Sport(id, nom, coach)
3 Seance(id, id_adherent, id_sport, jour)

```

`Seance.id_adherent` est une clé étrangère vers `Adherent.id`, et `Seance.id_sport` une clé étrangère vers `Sport.id`. Le nom de l'adhérent n'est désormais stocké qu'une seule fois, quel que soit le nombre de sports pratiqués.

Corrigé

Q1.

```

1 SELECT titre FROM Film WHERE realisateur = 'Miyazaki';

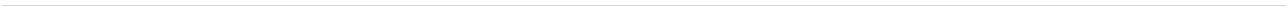
```

Q2.

```

1 SELECT titre FROM Film WHERE duree > 100;

```



4 Histoire de l'informatique

4.1 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.2 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.

- L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

4.3 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (reseau de reseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.4 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.
- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Méthodes

- ▶ Méthode directe de travail sur le sujet.

Exercice 1 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 2 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 3 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 4 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 5 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 6 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de généralité.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 7 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 8 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génèrite.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 9 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 10 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génèrite.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 11 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 12 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génèrite.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 13 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- Mise en service du réseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs à programme enregistré.

Exercice 14 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de génèrite.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

Exercice 15

Remettre dans l'ordre chronologique les evenements suivants, en precisant l'annee de chacun :

- Mise en service du reseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs a programme enregistre.

Exercice 16

Un smartphone moderne peut executer des milliers d'applications differentes sans que son materiel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'evolution du role du materiel vers plus de generite.
2. Citer deux couches logicielles (parmi systeme d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilite, et expliquer brievement le role de chacune.

Exercice 17

Remettre dans l'ordre chronologique les evenements suivants, en precisant l'annee de chacun :

- Mise en service du reseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs a programme enregistre.

Exercice 18

Un smartphone moderne peut executer des milliers d'applications differentes sans que son materiel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'evolution du role du materiel vers plus de generite.
2. Citer deux couches logicielles (parmi systeme d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilite, et expliquer brievement le role de chacune.

Exercice 19

Remettre dans l'ordre chronologique les evenements suivants, en precisant l'annee de chacun :

- Mise en service du reseau ARPANET.
- Formalisation de la machine universelle par Alan Turing.
- Construction des premiers ordinateurs a programme enregistre.

Exercice 20

Un smartphone moderne peut executer des milliers d'applications differentes sans que son materiel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'evolution du role du materiel vers plus de generite.
2. Citer deux couches logicielles (parmi systeme d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilite, et expliquer brievement le role de chacune.

Exercice 21

Remettre dans l'ordre chronologique les evenements suivants, en precisant l'annee de chacun :

- Mise en service du reseau ARPANET.

- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs a programme enregistre.

Exercice 22 ♦♦

Un smartphone moderne peut executer des milliers d'applications differentes sans que son materiel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'evolution du role du materiel vers plus de generite.
2. Citer deux couches logicielles (parmi systeme d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilite, et expliquer brievement le role de chacune.

Exercice 23 ♦

Remettre dans l'ordre chronologique les evenements suivants, en precisant l'annee de chacun :

- ▶ Mise en service du reseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs a programme enregistre.

Exercice 24 ♦♦

Un smartphone moderne peut executer des milliers d'applications differentes sans que son materiel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'evolution du role du materiel vers plus de generite.
2. Citer deux couches logicielles (parmi systeme d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilite, et expliquer brievement le role de chacune.

QCM d'auto-evaluation — Histoire de l'informatique

Pour chaque question, une seule reponse est exacte. Entourez la lettre correspondante.

C1 — Repères chronologiques

1. [Q1] Le concept de machine universelle est formalise par Alan Turing en :
 - A. 1936.
 - B. 1948.
 - C. 1969.
 - D. 1991.
2. [Q2] ARPANET, ancetre d'Internet, est mis en service en :
 - A. 1948.
 - B. 1969.
 - C. 1989.
 - D. 2000.

C2 — Évolution logiciel/matériel

3. [Q3] Depuis les debuts de l'informatique, le role du materiel a evolue vers :

- A. plus de specialisation.
- B. plus de generite (materiel generique, logique portee par le logiciel).
- C. une disparition progressive.
- D. aucune evolution notable.

4. [Q4] Internet et le World Wide Web :

- A. designent exactement la meme chose.
- B. sont deux reseaux physiques distincts et independants.
- C. le Web est un service qui fonctionne au-dessus du reseau Internet.
- D. Internet est un service qui fonctionne au-dessus du Web.

Corrigé

Q1. Machine universelle de Turing : 1936. Premiers ordinateurs a programme enregistre : 1948. ARPANET : 1969.

Q2. En 1936, les notions de machine, algorithme, langage et information sont pour la premiere fois pensees comme un tout coherent (concept de machine universelle, capable d'executer tout algorithme calculable).

Q3. Exemple accepte : diversification en telephones, tablettes, montres connectees, objets connectes, fermes de calcul (toute reponse coherente avec le cours est valable).

Corrigé

Q1. Aux debuts de l'informatique, le materiel portait directement la logique de calcul (cablages specifiques). Aujourd'hui, un meme materiel generique (smartphone, ordinateur) execute des logiques tres variees portees par le logiciel : le role du materiel a evolue vers plus de generite.

Q2. Le systeme d'exploitation gere le partage des ressources materielles (memoire, processeur, peripheriques) entre plusieurs programmes, ajoutant une couche logicielle entre l'utilisateur/les applications et le materiel, ce qui permet a ce dernier de rester generique.

Évaluation A — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques

(10 points)

1. (4 pts) Associer chacun des evenements suivants a son annee : machine universelle de Turing ; premiers ordinateurs a programme enregistre ; mise en service d'ARPANET.
2. (3 pts) Expliquer en une phrase pourquoi 1936 est consideree comme une date charniere dans l'histoire de l'informatique.
3. (3 pts) Citer un exemple de diversification des formes d'ordinateurs depuis les annees 1980.

Exercice 2 — Évolution logiciel/matériel

(10 points)

1. (5 pts) Expliquer, avec un exemple, comment le role du materiel a evolue depuis les debuts de l'informatique.
2. (5 pts) Expliquer le role d'un systeme d'exploitation dans cette evolution.

Corrigé

Q1. 1936, par Alan Turing.

Q2. 1948.

Q3. Exemple accepté : invention du World Wide Web par Tim Berners-Lee (1989-1991), qui popularise l'usage grand public d'Internet.

Corrigé

Q1. Dans les années 1950, le matériel portait directement la logique de calcul (cablages, code machine) : peu de flexibilité, chaque machine était quasi dédiée à une tâche. Aujourd'hui, le matériel est générique (processeurs polyvalents) et c'est le logiciel installé qui détermine entièrement le comportement de la machine, permettant une flexibilité quasi illimitée.

Q2. Exemple accepté : une console de jeux ou un ordinateur personnel peut exécuter des milliers de jeux ou d'applications très différents sans modification matérielle, uniquement par installation de logiciels différents.

Évaluation B — Histoire de l'informatique

Durée : 30 minutes. Documents interdits.

Barème indicatif : Ex. 1 : 10 pts (repères chronologiques) ; Ex. 2 : 10 pts (évolution logiciel/matériel)

Exercice 1 — Repères chronologiques

(10 points)

1. (4 pts) Quelle année marque la formalisation du concept de machine universelle, et par qui ?
2. (3 pts) En quelle année les premiers ordinateurs à programme enregistré sont-ils construits ?
3. (3 pts) Citer un événement marquant du développement d'Internet après 1969 (par exemple lié au World Wide Web).

Exercice 2 — Évolution logiciel/matériel

(10 points)

1. (5 pts) Expliquer la différence entre l'informatique des années 1950 (matériel dominant) et l'informatique actuelle (logiciel dominant).
2. (5 pts) Donner un exemple concret (autre que celui du cours) illustrant cette évolution.

4.5 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ 1936 : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ 1945 : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ 1948 : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.6 Évolution des rôles du logiciel et du matériel

♦ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.
- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

♦ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs a programme enregistré.
3. **1969** : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs a programme enregistré.
3. **1969** : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs a programme enregistré.
3. **1969** : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.

2. 1948 : construction des premiers ordinateurs a programme enregistré.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistré.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistré.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le materiel (processeur, memoire, ecran) reste identique quelle que soit l'application executee : c'est le logiciel installe qui determine le comportement du smartphone, illustrant un materiel devenu generique et polyvalent.

Q2. Le **systeme d'exploitation** gere l'accès aux ressources materielles (memoire, processeur, peripheriques) et fournit une interface commune aux applications. Le **navigateur** permet d'executer des applications web (ecrites en HTML/CSS/JavaScript) directement, sans installation dediee, ajoutant une couche d'abstraction supplementaire au-dessus du systeme d'exploitation.

Corrigé

1. 1936 : Alan Turing formalise le concept de machine universelle.
2. 1948 : construction des premiers ordinateurs a programme enregistré.
3. 1969 : mise en service du reseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

Corrigé

1. **1936** : Alan Turing formalise le concept de machine universelle.
2. **1948** : construction des premiers ordinateurs à programme enregistré.
3. **1969** : mise en service du réseau ARPANET.

Corrigé

Q1. Le matériel (processeur, mémoire, écran) reste identique quelle que soit l'application exécutée : c'est le logiciel installé qui détermine le comportement du smartphone, illustrant un matériel devenu générique et polyvalent.

Q2. Le **système d'exploitation** gère l'accès aux ressources matérielles (mémoire, processeur, périphériques) et fournit une interface commune aux applications. Le **navigateur** permet d'exécuter des applications web (écrites en HTML/CSS/JavaScript) directement, sans installation dédiée, ajoutant une couche d'abstraction supplémentaire au-dessus du système d'exploitation.

5 Langages, récursivité et paradigmes de programmation

5.1 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait **le contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.2 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```
1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n <= 1:
7         return n
```

```
8 return fib_naif(n - 1) + fib_naif(n - 2)
```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémorisation).

ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.3 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```
1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (`.py`) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
```

```

2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)

```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d’être mise à jour, tant que le comportement observable reste le même).

5.4 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d’une liste, dans trois styles différents.

Style impératif (une suite d’instructions qui modifient un état) :

```

1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total

```

Style fonctionnel (composition de fonctions, sans modifier d’état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```


◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions `map`, `filter`, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.5 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```
1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres
```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```
1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)
```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 ≤ indice < len(liste)
```

```
3 return 0 ≤ indice < len(liste)
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 def somme_avec_bug(valeurs):
2     sum = 0          # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum       # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.6 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction `s_arrete(P, e)` qui renvoie toujours correctement `True` si le programme `P` s'arrête sur l'entrée `e`, et `False` sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait **le contraire** de ce que prédit `s_arrete` :

- ▶ si `s_arrete(paradoxe, paradoxe)` vaut `True` (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si `s_arrete(paradoxe, paradoxe)` vaut `False` (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.7 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```

1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n ≤ 1:
7         return n
8     return fib_naif(n - 1) + fib_naif(n - 2)

```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.8 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```

1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)

```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (.py) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via import. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module geometrie.py regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)
```



ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d'être mise à jour, tant que le comportement observable reste le même).

5.9 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d'une liste, dans trois styles différents.

Style impératif (une suite d'instructions qui modifient un état) :

```
1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total
```

Style fonctionnel (composition de fonctions, sans modifier d'état) :

```
1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)
```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_paires(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un **même** programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.10 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```

1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres

```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```

1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)

```

⚠ ERREUR FRÉQUENTE

Croire que $b = a$ crée une copie de la liste a . En Python, cette instruction fait seulement pointer b vers le **même objet** que a : modifier b (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi a . Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit vérifier  $0 \leq \text{indice} < \text{len}(\text{liste})$ 
3     return  $0 \leq \text{indice} < \text{len}(\text{liste})$ 
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 def somme_avec_bug(valeurs):
2     sum = 0           # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum        # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.11 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout

autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait le **contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, s_arrete se trompe sur paradoxe appliqué à lui-même : une telle fonction s_arrete ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.12 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
```



```

2  if chaine == "":
3      return 0
4  premier = 1 if chaine[0] == caractere else 0
5  return premier + compter(chaine[1:], caractere)

```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```

1  compteur_appels = 0
2
3  def fib_naif(n):
4      global compteur_appels
5      compteur_appels += 1
6      if n ≤ 1:
7          return n
8      return fib_naif(n - 1) + fib_naif(n - 2)

```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémorisation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.13 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```

1  import statistics
2
3  notes = [12, 15, 8, 17, 14]
4  moyenne = statistics.mean(notes)

```

```

5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)

```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (.py) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```

1 def aire_rectangle(largeur, hauteur):
2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10    """Renvoie le perimetre d'un rectangle."""
11    return 2 * (largeur + hauteur)

```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d'être mise à jour, tant que le comportement observable reste le même).

5.14 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d'une liste, dans trois styles différents.

Style impératif (une suite d'instructions qui modifient un état) :

```

1 def somme_carres_pairs_imperatif(liste):
2     total = 0

```

```

3   for x in liste:
4       if x % 2 == 0:
5           total += x ** 2
6   return total

```

Style fonctionnel (composition de fonctions, sans modifier d'état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions `map`, `filter`, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.15 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```

1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)      # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres

```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```

1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a          # b designe le MEME objet liste que a, pas une copie
6 vider(b)

```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```

1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 ≤ indice < len(liste)
3     return 0 ≤ indice < len(liste)

```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```

1 def somme_avec_bug(valeurs):
2     sum = 0          # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum       # fonctionne ici, mais sum() natif est devenu inaccessible

```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

5.16 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```

1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":

```

```

6     accumulateur *= valeur
7     return accumulateur

```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait **le contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, s_arrete se trompe sur paradoxe appliqué à lui-même : une telle fonction s_arrete ne peut donc pas exister pour **tous** les cas.

ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

Méthodes

- ▶ Méthode directe de travail sur le sujet.

Exercice 1 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 2 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 3

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 4

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 5

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 6

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 7 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longts_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longts_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 8 ♦

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 9 ♦

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 10 ♦

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 11 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```

1 def mots_longes_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat

```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longes_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 12

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```

1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 13

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 14

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 15

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```

1 def mots_longes_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat

```


1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 16

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 17

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 18

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 19

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 20

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 21

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 22

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 23

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 24

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
```

```

3     return bulletin
4
5     bulletin_A = []
6     bulletin_B = ajouter_note(bulletin_A, 15)
7     bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 25

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 26

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 27

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```

1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat

```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 28

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```

1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5     bulletin_A = []
6     bulletin_B = ajouter_note(bulletin_A, 15)
7     bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 29

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récur­sifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 30

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 31

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 32

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 33 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 34 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 35 ◆◆

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 36 ◆

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 37 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 38

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 39

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 40

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 41

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 42

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.

- Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
- La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 43

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

- Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
- Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 44

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

- Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
- Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 45

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif `n` (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récurifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 46

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

- Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
- Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
- La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 47 ♦♦

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longts_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) ≥ longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longts_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 48 ♦

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```
1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)
```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

Exercice 49 ♦

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 50 ♦

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

QCM d'auto-évaluation — Langages et programmation

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C01C — Problème de l'arrêt

1. [Q1] Le problème de l'arrêt étant indécidable signifie que :
- A. aucun programme ne s'arrête jamais.
 - B. aucun programme unique ne peut décider, pour tous les programmes et toutes les entrées, si l'exécution s'arrête.
 - C. on ne peut jamais prouver qu'un programme particulier s'arrête.
 - D. ce problème n'existe qu'en Python.

C02B — Analyse de récursivité

2. [Q2] Un grand nombre d'appels dans l'arbre d'appels d'une fonction récursive signifie que :
- A. la fonction ne termine jamais.
 - B. la fonction termine, mais potentiellement avec un coût élevé.
 - C. il y a forcément une erreur dans le code.
 - D. le cas de base est mal écrit.

C03A — API et bibliothèques

3. [Q3] L'API d'une bibliothèque désigne :
- A. son code source complet.
 - B. l'ensemble des fonctions et paramètres qu'elle expose à l'utilisateur.
 - C. sa vitesse d'exécution.
 - D. le nom de son auteur.

C04A — Paradigmes

4. [Q4] Un même programme Python peut mobiliser :
- A. un seul paradigme, imposé par le langage.
 - B. plusieurs paradigmes à la fois (impératif, fonctionnel, objet).
 - C. uniquement le paradigme objet.
 - D. aucun paradigme particulier.

C05 — Débogage

5. [Q5] Comparer deux nombres flottants avec l'opérateur `==` est risqué car :
- A. Python interdit cette comparaison.
 - B. les flottants ne représentent pas exactement toutes les valeurs décimales.
 - C. cela ne fonctionne qu'avec des entiers.
 - D. les flottants sont toujours égaux entre eux.
6. [Q6] Écrire `b = a` où `a` est une liste :
- A. crée une copie indépendante de `a`.
 - B. fait pointer `b` vers le même objet que `a`.
 - C. provoque une erreur.
 - D. vide la liste `a`.

Corrigé

Q1. Le code source d'un programme est une suite de caractères, stockable et manipulable comme n'importe quelle donnée : par exemple, un interpréteur lit le code d'un autre programme comme une donnée en entrée, pour l'exécuter instruction par instruction.

Q2. Si un programme $s_arrete(P, e)$ pouvait toujours décider correctement si P s'arrête sur e , on pourrait construire un programme paradoxe qui fait exactement le contraire de ce que s_arrete (paradoxe, paradoxe) prédit pour lui-même : s_arrete se tromperait alors nécessairement sur ce cas précis, ce qui contredit l'hypothèse qu'il décide correctement dans tous les cas. Un tel programme ne peut donc pas exister.

Corrigé

Q1.

```
1 def puissance(base, exposant):
2     if exposant == 0:
3         return 1
4     return base * puissance(base, exposant - 1)
```

Q2. $puissance(2, 4)$ appelle $puissance(2, 3)$, qui appelle $puissance(2, 2)$, qui appelle $puissance(2, 1)$, qui appelle $puissance(2, 0)$: ce dernier atteint le cas de base ($exposant == 0$) et renvoie 1 directement. Les résultats remontent ensuite en cascade : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$.

Évaluation A — Calculabilité et récursivité

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (programme comme donnée, problème de l'arrêt) ; Ex. 2 : 12 pts (récursivité)

Exercice 1 — Programme comme donnée et calculabilité

(8 points)

- (4 pts) Expliquer, avec un exemple, en quoi un programme est aussi une donnée.
- (4 pts) Résumer en quelques phrases l'argument montrant que le problème de l'arrêt est indécidable (sans formalisme, l'idée du raisonnement par l'absurde suffit).

Exercice 2 — Récursivité

(12 points)

- (6 pts) Écrire une fonction récursive $puissance(base, exposant)$ qui calcule $base^{exposant}$ pour un exposant entier positif ou nul (sans utiliser l'opérateur $**$).
- (6 pts) Décrire l'arbre d'appels de $puissance(2, 4)$: quels appels successifs sont déclenchés, et quel est le cas de base atteint ?

Corrigé

```
1 def est_premier(n):
2     """Indique si l'entier n (n > 1) est premier.
3
4     Parametre : n, entier strictement superieur a 1.
5     Renvoie : True si n est premier, False sinon.
6     """
7     for d in range(2, n):
8         if n % d == 0:
9             return False
10    return True
11
12 print(est_premier(7)) # True
13 print(est_premier(8)) # False
```

Corrigé

Q1.

```

1 def carres_positifs_fonctionnel(valeurs):
2     return [v ** 2 for v in valeurs if v > 0]

```

Q2. La fonction native masquée est `sum`. Le code fonctionne malgré tout *ici* car la variable locale `sum` n'est utilisée que comme accumulateur, sans jamais appeler la fonction native `sum(...)` dans le reste de la fonction. Le risque apparaîtrait si l'on avait besoin d'appeler `sum(...)` plus loin dans le même bloc : l'appel échouerait car `sum` désignerait alors un entier, plus une fonction.

Q3. `resultat == 0.1` renvoie `False` : à cause des arrondis internes de la représentation binaire des flottants, $0.3 - 0.2$ ne vaut pas exactement 0.1 (l'écart est de l'ordre de 10^{-17}). Comparer deux flottants avec `==` est risqué pour cette raison ; il faut comparer leur écart à une tolérance près : `abs(resultat - 0.1) < 1e-9`.

Évaluation B — Modules, paradigmes et débogage

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (module documenté) ; Ex. 2 : 12 pts (paradigmes, débogage)

Exercice 1 — Créer un module documenté

(8 points)

- (5 pts) Écrire une fonction `est_premier(n)`, destinée à un module `arithmetique.py`, qui renvoie `True` si l'entier n (supérieur à 1) est premier, `False` sinon. Ajouter une docstring décrivant son rôle, son paramètre et sa valeur de retour.
- (3 pts) Tester la fonction sur $n = 7$ (premier) et $n = 8$ (non premier).

Exercice 2 — Paradigmes et débogage

(12 points)

- (4 pts) Réécrire en style fonctionnel (une liste en compréhension) la fonction impérative suivante :

```

1 def carres_positifs_imperatif(valeurs):
2     resultat = []
3     for v in valeurs:
4         if v > 0:
5             resultat.append(v ** 2)
6     return resultat

```

- (4 pts) Le code suivant contient un bug de nommage :

```

1 def calculer_moyenne(notes):
2     sum = 0
3     for n in notes:
4         sum += n
5     return sum / len(notes)

```

Quelle fonction native est masquée ? Ce code fonctionne-t-il malgré tout pour calculer une moyenne simple ? Justifier.

- (4 pts) Le code suivant compare deux résultats de calculs flottants avec `==`. Que renvoie-t-il, pourquoi est-ce risqué de comparer ainsi, et comment corriger ?

```

1 resultat = 0.3 - 0.2
2 print(resultat == 0.1)

```

Exercice 51

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 52

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 53

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 54 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 55 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 56 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<= n` au lieu de **stricte** `< n` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 57

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 58

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 59

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 60 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 61 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Exercice 62 ◆

Une fonction censée vérifier qu'un indice est valide pour une liste de longueur n est écrite ainsi :

```
1 def indice_valide_bug(indice, n):
2     return 0 ≤ indice ≤ n
```

Pour une liste de longueur $n = 5$ (indices valides : 0 à 4), que renvoie `indice_valide_bug(5, 5)` ? Est-ce correct ? Corriger la fonction.

Corrigé

`indice_valide_bug(5, 5)` renvoie `True`, ce qui est **incorrect** : pour une liste de longueur 5, les indices valides sont 0, 1, 2, 3, 4 (indice 5 est hors limites, il provoquerait un `IndexError`). L'erreur vient de l'inégalité **large** `<=` au lieu de **stricte** `<` :

```
1 def indice_valide_corrige(indice, n):
2     return 0 ≤ indice < n
```

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
```



```
3     return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_`

chiffres(1234), l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
6 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]
3
4 mots = ["chat", "ordinateur", "cle", "programmation"]
5 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
6 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2

```

```

3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6

```

```

5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:

```

```

7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_longfs_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_longfs_imperatif(mots, 6) == mots_longfs_fonctionnel(mots, 6)) # True
3 print(mots_longfs_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (résultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```


`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
3 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. `bulletin_A`, `bulletin_B` et `bulletin_C` désignent tous les **trois le même objet liste**, qui contient finalement `[15, 12]`. Ce n'est **pas** le résultat attendu : `append` modifie la liste **en place** (effet de bord), donc chaque appel modifie le même `bulletin_A` au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur `+` sur des listes crée une **nouvelle** liste (concaténation), sans modifier `bulletin` : `bulletin_A2` reste `[]`, `bulletin_B2` vaut `[15]` et `bulletin_C2` vaut `[12]`, chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est `n < 10` (un seul chiffre) : la fonction renvoie `n` directement. Le cas général sépare le dernier chiffre (`n % 10`) du reste du nombre (`n // 10`), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```
1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
3 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']
```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. bulletin_A, bulletin_B et bulletin_C désignent tous les **trois le même objet liste**, qui contient finalement [15, 12]. Ce n'est **pas** le résultat attendu : append modifie la liste **en place** (effet de bord), donc chaque appel modifie le même bulletin_A au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```
1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]
```

L'opérateur + sur des listes crée une **nouvelle** liste (concaténation), sans modifier bulletin : bulletin_A2 reste [], bulletin_B2 vaut [15] et bulletin_C2 vaut [12], chacun indépendant des autres.

Corrigé

```
1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)
```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

Corrigé

```
1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"
```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.

Corrigé

```

1 def mots_long_fonctionnel(mots, longueur_min):
2     return [mot.upper() for mot in mots if len(mot) ≥ longueur_min]

1 mots = ["chat", "ordinateur", "cle", "programmation"]
2 print(mots_long_imperatif(mots, 6) == mots_long_fonctionnel(mots, 6)) # True
3 print(mots_long_fonctionnel(mots, 6)) # ['ORDINATEUR', 'PROGRAMMATION']

```

Les deux versions produisent le même résultat : la version fonctionnelle exprime en une seule expression ce que la version impérative construit pas à pas avec une variable d'accumulation (resultat) modifiée par des instructions successives.

Corrigé

Q1. bulletin_A, bulletin_B et bulletin_C désignent tous les **trois le même objet liste**, qui contient finalement [15, 12]. Ce n'est **pas** le résultat attendu : append modifie la liste **en place** (effet de bord), donc chaque appel modifie le même bulletin_A au lieu de créer un bulletin séparé pour chaque élève.

Q2.

```

1 def ajouter_note_corrige(bulletin, note):
2     return bulletin + [note]

```

L'opérateur + sur des listes crée une **nouvelle** liste (concaténation), sans modifier bulletin : bulletin_A2 reste [], bulletin_B2 vaut [15] et bulletin_C2 vaut [12], chacun indépendant des autres.

Corrigé

```

1 def somme_chiffres(n):
2     if n < 10:
3         return n
4     return n % 10 + somme_chiffres(n // 10)

```

Le cas de base est $n < 10$ (un seul chiffre) : la fonction renvoie n directement. Le cas général sépare le dernier chiffre ($n \% 10$) du reste du nombre ($n // 10$), et additionne récursivement. Pour `somme_chiffres(1234)`, l'arbre d'appels est `somme_chiffres(1234) → somme_chiffres(123) → somme_chiffres(12) → somme_chiffres(1)` : 4 appels au total (le dernier atteint le cas de base).

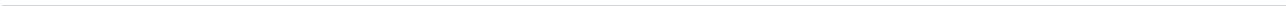
Corrigé

```

1 import math
2
3 print(math.hypot(3, 4)) # 5.0
4 print(math.gcd(48, 18)) # 6
5
6 try:
7     math.sqrt("4")
8 except TypeError as erreur:
9     print(erreur) # "must be real number, not str"

```

`math.sqrt` attend un nombre (entier ou flottant), pas une chaîne de caractères : appeler `math.sqrt("4")` lève une `TypeError`, même si "4" « ressemble » à un nombre. Il faudrait d'abord convertir explicitement avec `math.sqrt(int("4"))` ou `math.sqrt(float("4"))`.



6 Structures de données

6.1 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **operations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```
1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînee:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None
```

EXEMPLE Ces deux implementations offrent **exactement la meme interface** (memes trois methodes, meme comportement observable), mais des structures de donnees internes totalement differentes (tableau dynamique contre liste chaine).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : apres avoir empiler 1, 2, 3, depiler doit renvoyer 3), pas sur les details internes (comme la structure exacte de `_donnees`). Cela permet au meme jeu de tests de valider plusieurs implementations.

6.2 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modele qui decrit la structure (**attributs**, l'etat) et le comportement (**methodes**) des objets qui en sont des **instances**. En Python, une classe se definit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnee associee a un objet (etat), accedee par objet.attribut.
 - **Methode** : fonction associee a une classe, appelee par objet.methode(...). Le premier parametre `self` designe l'objet lui-meme (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier cree a partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier self dans la définition d'une méthode. Toute méthode d'instance doit avoir self comme premier parametre (même si on ne l'appelle jamais explicitement lors de l'appel objet.methode()) : Python le fournit automatiquement.

6.3 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier element ajoute est le premier retire. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier element ajoute est le premier retire. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (collections.deque est preferable en pratique pour l'efficacite, mais une liste suffit pour illustrer le principe) :

```
1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11     def est_vide(self):
12         return len(self._donnees) == 0
```

COURS

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, a l'oppose du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix depend des **operations dominantes** de la situation a modeliser :

- ▶ **Pile** : annulation d'actions (undo), evaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de taches dans l'ordre d'arrivee, parcours en largeur, files d'attente.
- ▶ **Liste** : acces par position (indice), parcours sequentiel.
- ▶ **Dictionnaire** : acces direct par cle (recherche rapide en moyenne, independante de la taille), sans notion d'ordre de position.

ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** element (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** element (comportement FIFO, utile pour une file, mais couteux car il faut decaler tous les elements restants).

6.4 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitue d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un noeud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de noeuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction récursive sur un arbre. Toute fonction récursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la récursion atteint un sous-arbre vide.

6.5 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **aretes**. Un graphe est **orienté** si ses aretes ont un sens (representees par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arete de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le meme graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) represente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j ou $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arete (i, j) , mais occupe une memoire en n^2 meme si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une memoire proportionnelle au nombre d'aretes reellement presentes, plus adaptees aux graphes creux, mais tester l'existence d'une arete precise peut necessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand meme apparaitre. Dans la representation par listes de successeurs, un sommet isole ou puits (sans arete sortante) doit etre present dans

le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.6 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : `empiler(valeur)`, `depiler()`, `est_vide()`.

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```

1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînée:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None

```

EXEMPLE Ces deux implémentations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empilé 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.7 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-clé `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier `self` dans la définition d'une méthode. Toute méthode d'instance doit avoir `self` comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel `objet.methode()`) : Python le fournit automatiquement.

6.8 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (`collections.deque` est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```
1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Ici, "A" est le premier entré et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par clé (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre `.pop()` et `.pop(0)`. Pour une liste Python, `.pop()` retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que `.pop(0)` retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.9 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un nœud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles** • La **taille** d'un arbre est son nombre total de noeuds.
- La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
- Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.10 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **aretes**. Un graphe est **orienté** si ses aretes ont un sens (représentées par des fleches), **non orienté** sinon.

- ◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe à n sommets numérotés de 0 à $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ où la case (i, j) vaut 1 (ou True) s'il existe une arete de i vers j , 0 sinon.

Exemple. Graphe orienté à 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représenté par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j où $\text{matrice}[i][j] = 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une mémoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une mémoire proportionnelle au nombre d'arêtes réellement présentes, plus adaptées aux graphes creux, mais tester l'existence d'une arête précise peut nécessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaître. Dans la représentation par listes de successeurs, un sommet isolé ou puits (sans arête sortante) doit être présent dans le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.11 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : `empiler(valeur)`, `depiler()`, `est_vide()`.

Implementation 1 — avec une liste Python (fin de liste = sommet) :

```

1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implementation 2 — avec une liste chainee (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaine:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None

```

EXEMPLE Ces deux implementations offrent **exactement la meme interface** (memes trois methodes, meme comportement observable), mais des structures de donnees internes totalement differentes (tableau dynamique contre liste chainee).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : apres avoir empiler 1, 2, 3, depiler doit renvoyer 3), pas sur les details internes (comme la structure exacte de `_donnees`). Cela permet au meme jeu de tests de valider plusieurs implementations.

6.12 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

⚠ ERREUR FRÉQUENTE

Oublier `self` dans la définition d'une méthode. Toute méthode d'instance doit avoir `self` comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel `objet.methode()`) : Python le fournit automatiquement.

6.13 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Interface : empiler, dépiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (`collections.deque` est preferable en pratique pour l'efficacite, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, a l'oppose du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix depend des **operations dominantes** de la situation a modeliser :

- ▶ **Pile** : annulation d'actions (undo), evaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de taches dans l'ordre d'arrivee, parcours en largeur, files d'attente.
- ▶ **Liste** : acces par position (indice), parcours sequentiel.
- ▶ **Dictionnaire** : acces direct par cle (recherche rapide en moyenne, independante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre `.pop()` et `.pop(0)`. Pour une liste Python, `.pop()` retire et renvoie le **dernier** element (comportement LIFO, utile pour une pile), tandis que `.pop(0)` retire et renvoie le **premier** element (comportement FIFO, utile pour une file, mais couteux car il faut decaler tous les elements restants).

6.14 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitue d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-memes des arbres binaires. Une **feuille** est un noeud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de noeuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine a une feuille (hauteur 0 pour un arbre reduit a une seule feuille, hauteur -1 ou non definie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.15 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **aretes**. Un graphe est **orienté** si ses aretes ont un sens (representées par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arete de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```

1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]

```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le meme graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) represente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j ou $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arete (i, j) , mais occupe une memoire en n^2 meme si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une memoire proportionnelle au nombre d'aretes reellement presentes, plus adaptees aux graphes creux, mais tester l'existence d'une arete precise peut necessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand meme apparaitre. Dans la representation par listes de successeurs, un sommet isole ou puits (sans arete sortante) doit etre present dans le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

6.16 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de donnees est l'ensemble des **operations** qu'elle propose (ce que l'on peut faire), **sans preciser comment** elles sont realisees. Une **implementation** est une facon concrete de coder ces operations (avec quelles donnees internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implementation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
```

```

8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Implementation 2 — avec une liste chaînée (debut = sommet) :

```

1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaine:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None

```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empilé 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.17 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```

1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):

```

```

7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10         self.x = self.x + dx
11         self.y = self.y + dy

```

Utilisation :

```

1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5

```

- ◆ **PROPRIÉTÉ Vocabulaire**
- **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre self désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple p est une instance de Point).

⚠ ERREUR FRÉQUENTE

Oublier self dans la définition d'une méthode. Toute méthode d'instance doit avoir self comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel objet.methode()) : Python le fournit automatiquement.

6.18 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Interface : empiler, dépiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retiré. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (collections.deque est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11     def est_vide(self):
12         return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

⚠ ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.19 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un nœud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de nœuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de nœuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16

```

```

17 def nb_feuilles(a):
18     if a is None:
19         return 0
20     if a.gauche is None and a.droit is None:
21         return 1
22     return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

Méthodes

- Méthode directe de travail sur le sujet.

Exercice 1 ♦♦

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface empiler, depiler, est_vide) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 2 ♦

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 3 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 4 ♦♦

On donne le graphe orienté à 4 sommets $(0, 1, 2, 3)$ représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 5

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 6

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- un constructeur prenant un titulaire et un solde initial (par default 0) ;
- une methode `deposer(montant)` qui augmente le solde ;
- une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 7

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 8

On donne le graphe oriente a 4 sommets (0,1,2,3) represente par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.

3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 9 ♦♦

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 10 ♦

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par default 0) ;
- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 11 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 12 ♦♦

On donne le graphe oriente a 4 sommets (0, 1, 2, 3) represente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 13 ♦♦

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).

2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 14

Ecrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 15

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 16

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 17

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthésée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 18

Ecrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;

- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 19 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 20 ♦♦

On donne le graphe oriente a 4 sommets (0, 1, 2, 3) represente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 21 ♦♦

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 22 ♦

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par default 0) ;
- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 23 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 24

On donne le graphe oriente a 4 sommets $(0, 1, 2, 3)$ represente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 25

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 26

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- un constructeur prenant un titulaire et un solde initial (par default 0) ;
- une methode `deposer(montant)` qui augmente le solde ;
- une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 27

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 28

On donne le graphe oriente a 4 sommets $(0, 1, 2, 3)$ represente par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 29 ♦♦

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"({})"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 30 ♦

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par default 0) ;
- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 31 ♦♦

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 32 ♦♦

On donne le graphe oriente a 4 sommets (0,1,2,3) represente par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.

- En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 33

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
- Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
- Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 34

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- un constructeur prenant un titulaire et un solde initial (par default 0) ;
- une methode `deposer(montant)` qui augmente le solde ;
- une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 35

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

- En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
- Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
- Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 36

On donne le graphe oriente a 4 sommets (0, 1, 2, 3) represente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

- Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
- Le sommet 2 a-t-il des successeurs ? Interpreter.
- En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 37

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

- Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).

2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 38

Ecrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- une méthode `deposer(montant)` qui augmente le solde ;
- une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 39

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 40

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 41

On veut vérifier si une expression comportant des parenthèses `()`, crochets `[]` et accolades `{}` est **bien parenthésée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthésée) et `"([)]"` (mal parenthésée).

Exercice 42

Ecrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- un constructeur prenant un titulaire et un solde initial (par défaut 0) ;

- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 43

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 44

On donne le graphe oriente a 4 sommets (0, 1, 2, 3) represente par la matrice d'adjacence :

```
1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]
```

1. Lister toutes les aretes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpreter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la representation par listes de successeurs de ce graphe.

Exercice 45

On veut verifier si une expression comportant des parentheses `()`, crochets `[]` et accolades `{}` est **bien parenthesee** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptee a ce probleme (justifier par les operations dominantes).
2. Ecrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour resoudre ce probleme.
3. Tester votre fonction sur `"([{}])"` (bien parenthesee) et `"([)]"` (mal parenthesee).

Exercice 46

Ecrire une classe `Compte` representant un compte bancaire simplifie, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par default 0) ;
- ▶ une methode `deposer(montant)` qui augmente le solde ;
- ▶ une methode `retirer(montant)` qui diminue le solde, et leve une erreur (`ValueError`) si le montant demande depasse le solde disponible.

Tester votre classe avec un compte initial de 100, un depot de 50, puis un retrait de 30.

Exercice 47

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-meme un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans executer de code, predire la taille et la hauteur de cet arbre. Justifier.
3. Verifier votre prediction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 48

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

Exercice 49

On veut vérifier si une expression comportant des parenthèses (), crochets [] et accolades {} est **bien parenthésée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur "`{[()]}`" (bien parenthésée) et "`([)]`" (mal parenthésée).

Exercice 50

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demandé dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

QCM d'auto-évaluation — Structures de données

Pour chaque question, une seule réponse est exacte. Entourez la lettre correspondante.

C01B/C02A — Interface, classes

1. [Q1] L'interface d'une structure de données décrit :
 - A. comment les données sont stockées en mémoire.
 - B. les opérations disponibles, sans préciser leur réalisation.
 - C. le nombre d'octets utilisés.

D. le langage de programmation utilise.

2. [Q2] Dans une classe Python, le parametre self designe :
- A. la classe elle-meme.
 - B. l'objet sur lequel la methode est appelee.
 - C. le premier attribut de la classe.
 - D. rien de particulier, il est optionnel.

C03A/C03C — Piles, files, recherche

3. [Q3] Une pile fonctionne selon le principe :
- A. FIFO.
 - B. LIFO.
 - C. aleatoire.
 - D. trie.
4. [Q4] Rechercher une valeur associee a une cle dans un dictionnaire est, en moyenne, plus rapide que dans une liste car :
- A. le dictionnaire est toujours plus petit.
 - B. l'accès se fait directement via une table de hachage, independamment de la taille.
 - C. les dictionnaires sont tries automatiquement.
 - D. ce n'est pas vrai, c'est toujours identique.

C04B/C05B — Arbres, graphes

5. [Q5] La hauteur d'un arbre reduit a une seule feuille (racine sans enfant) vaut :
- A. -1.
 - B. 0.
 - C. 1.
 - D. indefinie.
6. [Q6] Dans une matrice d'adjacence d'un graphe a n sommets, la case (i, j) vaut 1 si :
- A. les sommets i et j ont le meme numero.
 - B. il existe une arete de i vers j .
 - C. le sommet i est une feuille.
 - D. $i + j = n$.

Corrigé

```

1 class Rectangle:
2     def __init__(self, largeur, hauteur):
3         self.largeur = largeur
4         self.hauteur = hauteur
5
6     def aire(self):
7         return self.largeur * self.hauteur
8
9     def perimetre(self):
10        return 2 * (self.largeur + self.hauteur)

```

Corrigé

Q1. Interface : empiler(v), depiler(), est_vide(). Principe LIFO : le dernier element empile est le premier depile.

Q2. Apres avoir empile 5,3,8,1, la pile est [5,3,8,1] (du bas vers le sommet). Depiler trois fois retire 1, puis 8, puis 3 (LIFO) : il reste [5].

Évaluation A — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 10 pts (programmer une classe); Ex. 2 : 10 pts (pile, LIFO)

Exercice 1 — Classe Rectangle

(10 points)

1. (4 pts) Ecrire une classe Rectangle avec un constructeur prenant largeur et hauteur.
2. (3 pts) Ajouter une methode aire() qui renvoie l'aire du rectangle.
3. (3 pts) Ajouter une methode perimetre() qui renvoie le perimetre du rectangle.

Exercice 2 — Pile

(10 points)

1. (4 pts) Rappeler l'interface d'une pile (trois methodes) et le principe LIFO.
2. (6 pts) On empile successivement 5,3,8,1, puis on depile trois fois. Donner, en le justifiant, le contenu final de la pile.

Corrigé

Q1. Un graphe modelise naturellement des elements (stations) en relation (lignes directes) : les stations sont les sommets, les lignes directes les aretes.

Q2. Si les lignes fonctionnent dans les deux sens, le graphe doit etre **non orienté** (une arete entre deux stations represente une liaison utilisable dans les deux sens, sans distinction de direction).

Corrigé

Q1. $0 \rightarrow 1, 0 \rightarrow 2, 2 \rightarrow 0$.

Q2. La liste associee au sommet 1 est vide : 1 n'a aucun successeur, c'est un puits du graphe (aucune arete sortante).

Q3.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 0],
4     [1, 0, 0],
5 ]
```

Évaluation B — Structures de données

Durée : 45 minutes. Ordinateur avec interpréteur Python autorisé.

Barème indicatif : Ex. 1 : 8 pts (modélisation); Ex. 2 : 12 pts (graphes, conversion)

Exercice 1 — Modélisation

(8 points)

Un reseau de transport relie des stations par des lignes directes.

1. (4 pts) Justifier qu'un graphe est une structure adaptée pour modéliser ce réseau.
2. (4 pts) Ce graphe doit-il être orienté ou non orienté si les lignes fonctionnent dans les deux sens ? Justifier.

Exercice 2 — Graphes, conversion

(12 points)

On donne un graphe orienté à 3 sommets (0, 1, 2) par ses listes de successeurs :

```
1 successeurs = {0: [1, 2], 1: [], 2: [0]}
```

1. (4 pts) Lister toutes les arêtes de ce graphe.
2. (4 pts) Le sommet 1 a-t-il des successeurs ? Qu'est-ce que cela signifie ?
3. (4 pts) Donner la matrice d'adjacence correspondant à ce graphe.

Exercice 51

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 52

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 53

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 54

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 55

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 56 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 57 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 58 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 59 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 60 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 61 ◆

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 62 ♦

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 63 ♦

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 64 ♦

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Exercice 65 ♦

On empile successivement les valeurs 10, 20, 30 dans une pile, puis on depile deux fois. Quelle est la valeur au sommet de la pile après ces opérations ?

Corrigé

Après avoir empilé 10, 20, 30, la pile est (du bas vers le sommet) [10, 20, 30]. Depiler une fois retire 30 (LIFO), depiler une deuxième fois retire 20. Il reste 10 au sommet.

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
```

```

7
8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15     def est_equilibre(expr):
16         p = Pile()
17         paires = {'(': ')', '[': ']', '{': '}'
18         for c in expr:
19             if c in '([{':
20                 p.empiler(c)
21             elif c in ')]}':
22                 if p.est_vide() or p.depiler() != paires[c]:
23                     return False
24         return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformement à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.


```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
```

```

3     self.titulaire = titulaire
4     self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 aretes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmee.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformement a la remarque du cours).

Corrigé

Q1. Une pile est adaptee car on doit toujours refermer le symbole ouvert le **plus recemment** en premier (comportement LIFO) : chaque symbole ouvrant est empile, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):

```

```

16 p = Pile()
17 paires = {'(': ')', '[': ']', '{': '}'
18 for c in expr:
19     if c in '([{':
20         p.empiler(c)
21     elif c in ')]}':
22         if p.est_vide() or p.depiler() != paires[c]:
23             return False
24     return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie `True` ; `est_equilibre("([)])")` renvoie `False` (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 aretes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{' :
20                p.empiler(c)
21            elif c in ')]}' :
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
```

```

10     if montant > self.solde:
11         raise ValueError("solde insuffisant")
12     self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaine de fils gauches), soit 4 aretes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmee.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformement a la remarque du cours).

Corrigé

Q1. Une pile est adaptee car on doit toujours refermer le symbole ouvert le **plus recemment** en premier (comportement LIFO) : chaque symbole ouvrant est empile, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15     def est_equilibre(expr):
16         p = Pile()
17         paires = {'(': ')', '[': ']', '{': '}' }
18         for c in expr:
19             if c in '([{':
20                 p.empiler(c)
21             elif c in ')]}':
22                 if p.est_vide() or p.depiler() != paires[c]:

```

```
23         return False
24     return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant
```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
```

```

5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}' }
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.


```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'}': '(', ']' : '[' , '}' : '{'}
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant
```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```
1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))
```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15     def est_equilibre(expr):
16         p = Pile()
17         paires = {'(': ')', '[': ']', '{': '}' }
18         for c in expr:
19             if c in '([{':
20                 p.empiler(c)
21             elif c in ')]}':
22                 if p.est_vide() or p.depiler() != paires[c]:
23                     return False
24         return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 aretes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmee.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1$, $0 \rightarrow 3$, $1 \rightarrow 2$, $3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformement a la remarque du cours).

Corrigé

Q1. Une pile est adaptee car on doit toujours refermer le symbole ouvert le **plus recemment** en premier (comportement LIFO) : chaque symbole ouvrant est empile, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13

```

```

14
15 def est_equilibre(expr):
16     p = Pile()
17     paires = {'(': ')', '[': ']', '{': '}'
18     for c in expr:
19         if c in '([{':
20             p.empiler(c)
21         elif c in ')]}':
22             if p.est_vide() or p.depiler() != paires[c]:
23                 return False
24     return p.est_vide()

```

Q3. `est_equilibre("({})")` renvoie True ; `est_equilibre("([)])"` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prediction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```

1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}

```

Le sommet 2, bien que sans successeur, apparait dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```

1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11    def est_vide(self):
12        return len(self._d) == 0
13
14
15    def est_equilibre(expr):
16        p = Pile()
17        paires = {'(': ')', '[': ']', '{': '}'
18        for c in expr:
19            if c in '([{':
20                p.empiler(c)
21            elif c in ')]}':
22                if p.est_vide() or p.depiler() != paires[c]:
23                    return False
24        return p.est_vide()

```

CORRIGÉS

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)])")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```

1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
5
6     def deposer(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant

```

Test : `c = Compte("Alice", 100)` ; après `c.deposer(50)`, `c.solde` vaut 150 ; après `c.retirer(30)`, `c.solde` vaut 120.

Corrigé

Q1.

```

1 arbre = Arbre(1, Arbre(2, Arbre(3, Arbre(4, Arbre(5)))))

```

Q2. L'arbre a 5 noeuds (1,2,3,4,5), donc **taille** = 5. Le plus long chemin racine-feuille passe par tous les noeuds (chaîne de fils gauches), soit 4 arêtes parcourues : **hauteur** = 4.

Q3. `taille(arbre)` renvoie 5, `hauteur(arbre)` renvoie 4 : la prédiction est confirmée.

Corrigé

Q1. En lisant les 1 de la matrice : $0 \rightarrow 1, 0 \rightarrow 3, 1 \rightarrow 2, 3 \rightarrow 2$.

Q2. La ligne 2 de la matrice ne contient que des 0 : le sommet 2 n'a **aucun successeur** (c'est un puits du graphe).

Q3.

```
1 successeurs = {0: [1, 3], 1: [2], 2: [], 3: [2]}
```

Le sommet 2, bien que sans successeur, apparaît dans le dictionnaire avec une liste vide (conformément à la remarque du cours).

Corrigé

Q1. Une pile est adaptée car on doit toujours refermer le symbole ouvert le **plus récemment** en premier (comportement LIFO) : chaque symbole ouvrant est empilé, et chaque symbole fermant doit correspondre au sommet de la pile.

Q2.

```
1 class Pile:
2     def __init__(self):
3         self._d = []
4
5     def empiler(self, v):
6         self._d.append(v)
7
8     def depiler(self):
9         return self._d.pop()
10
11     def est_vide(self):
12         return len(self._d) == 0
13
14
15     def est_equilibre(expr):
16         p = Pile()
17         paires = {'(': ')', '[': ']', '{': '}' }
18         for c in expr:
19             if c in '([{':
20                 p.empiler(c)
21             elif c in ')]}':
22                 if p.est_vide() or p.depiler() != paires[c]:
23                     return False
24         return p.est_vide()
```

Q3. `est_equilibre("([{}])")` renvoie True ; `est_equilibre("([)]")` renvoie False (le `)` rencontre alors que le sommet de la pile est `[`, qui ne correspond pas).

Corrigé

```
1 class Compte:
2     def __init__(self, titulaire, solde=0):
3         self.titulaire = titulaire
4         self.solde = solde
```

```
5
6     def deposter(self, montant):
7         self.solde = self.solde + montant
8
9     def retirer(self, montant):
10        if montant > self.solde:
11            raise ValueError("solde insuffisant")
12        self.solde = self.solde - montant
```

Test : `c = Compte("Alice", 100)` ; apres `c.deposer(50)`, `c.solde` vaut 150 ; apres `c.retirer(30)`, `c.solde` vaut 120.

Apprendre, s'entraîner, se tester, progresser : la collection Nexus

Réussite accompagne chaque élève jusqu'à la maîtrise.

- 📖 Un cours structuré en strates, avec la rubrique « À la machine » : chaque notion se reproduit au clavier.
- » Du code Python vérifié par exécution : aucune sortie de programme imprimée sans avoir tourné.
- ⚙ Des méthodes pas à pas avec exemples résolus commentés et pièges à éviter.
- ♦♦ Trois parcours d'exercices gradués et chronométrés, avec coups de pouce.
- ↺ QCM diagnostiques, auto-évaluation par compétences et sujets aux formats officiels.
- 🩹 Des fiches de remédiation ciblées, reliées aux erreurs réellement diagnostiquées.